N

The Backend

What happens behind NavigSA, in plain language

No prior knowledge needed. If you have ever used a spreadsheet and a library card, you already have the two ideas this rests on.

Database · Accounts · File storage · Security rules

FIRST, THE WORDS

Frontend and backend

Frontend - what you see

The screens, buttons, colours and forms.

It runs on your own phone or laptop, inside your browser.

It is good at showing things. It cannot be trusted to keep secrets, because anyone can open it up and look inside.

Backend - where things are kept

The information itself, and the rules about who may see it.

It runs somewhere else entirely, on computers you do not control.

It is what makes your data still be there tomorrow, on a different device.

A restaurant

The dining room is the frontend: the menu, the tables, the waiter taking your order. The kitchen and the store room are the backend. Guests never walk into the store room - and that is not rudeness, it is how the food stays safe.

THE BACKEND WE USE

We did not write a server. We configured one.

The old way

Write a server program yourself.

Find somewhere to run it.

Keep it patched, online and backed up, forever.

What we did

Use Supabase, which already provides all of it.

We describe the rules.

Supabase enforces them.

What it gives us

A database, to store information

An account system, for logging in

File storage, for uploaded documents

A rule engine, to police all three

Why this matters for a student project

There is no server of ours that can crash, run out of disk, or be broken into. The parts that are hardest to get right - storing passwords safely, issuing login tokens - are handled by people who do only that.

THE DATABASE

A table is just a spreadsheet

If you can read a spreadsheet

Columns are the kinds of information you keep. Rows are the individual records.

A database table is the same idea, with one addition: it can refuse to store nonsense, and it can refuse to show you someone else's row.

profiles

first_name	last_name	country
Gaffer	Kabala	Angola
Ana	Silva	Mozambique

one row per person who has an account

NavigSA keeps four tables

profiles - one row per person. applications - one row per university application. documents - one row per uploaded file. services - one row per provider in the marketplace.

HOW ROWS ARE CONNECTED

Everything carries a name tag

The user_id column

Every application and every document has a column recording which account it belongs to.

It is the equivalent of writing your name on your belongings before you leave them somewhere.

One small column - and every security rule in the system is built on it.

Gaffer's account

3 applications, 4 documents - all tagged with Gaffer's id

Ana's account

2 applications, 6 documents - all tagged with Ana's id

They share a table, never a view

Gaffer's and Ana's documents sit in the same table, side by side. What keeps them apart is not separate storage - it is the rule that reads the name tag before answering any question.

ACCOUNTS

What actually happens when you sign up

1	**You type**	An email address and a password on the signup form.
2	**It is scrambled**	Supabase never stores the password itself, only a scrambled version. Nobody - including us - can read it back.
3	**You prove the email**	A link is sent to that address. Until you click it, there is no account to sign in with.
4	**A profile appears**	The moment the account exists, the database automatically creates the matching profiles row.
5	**You sign in**	From then on, email and password get you a pass that keeps you logged in.

Why the email step exists

It proves the address belongs to you - so nobody can create an account in your name, and so a password reset has somewhere safe to go.

STAYING LOGGED IN

The wristband

What a token is

When you sign in, Supabase hands your browser a small signed pass called a token.

Your browser attaches it to every request it makes afterwards.

It says who you are. It is signed, so it cannot be forged or edited. It expires, and renews quietly in the background.

The festival wristband

You show identification once, at the gate.

After that you wear a wristband. Staff inside glance at the wristband rather than asking for your passport again and again.

You cannot draw one yourself that passes, and it stops working when the festival ends.

This is why closing the tab does not log you out

The token is kept safely in your browser, so returning to the site tomorrow finds you still signed in - until it expires or you choose to sign out.

THE DATABASE DOES SOME WORK BY ITSELF

Standing instructions

What a trigger is

A standing instruction attached to a table:

whenever this happens, also do that

It runs inside the database. It does not need the app to remember to ask.

```
-- our standing instruction

whenever a new account is created
  also create that person's
  profiles row
```

Why not just let the app do it

Because the app might not. It could crash between creating the account and creating the profile, leaving a person who can log in but has no profile to read. Someone could also create an account without going through our screens at all. Putting the instruction in the database means it happens every time, regardless of how the account was made - there is no path around it.

THE MOST IMPORTANT IDEA

Why can't someone just ask for everyone's data?

The honest question

Your browser holds a key to the database. Anyone can open the browser's developer tools and read that key. So what stops a curious person asking for every student's documents?

The answer

They can ask.

The database simply will not answer - it hands back an empty result.

The records office

You may walk up and ask for every file in the building. The clerk checks who you are and hands you only your own.

No amount of insisting changes that.

THE DIFFERENCE THAT MATTERS

A signpost or a locked door

Checking in the app only

The screen decides what to show you.

That is a sign on a door reading 'staff only'.

It works on people who read signs. It does nothing at all to someone who simply walks through.

Checking in the database

The store itself decides what to hand over.

That is a locked door.

Bypassing our screens gains nothing, because the refusal does not happen on the screen. It happens where the data actually lives.

NavigSA does both, and only one of them counts

The app hides pages you are not signed in for, because it makes the site pleasant to use. The database refuses rows you do not own, because that is what actually keeps the data safe.

THE RULE ITSELF

Three lines, in English underneath

```
create policy "read own documents"
  on documents for select
  using ( auth.uid() = user_id );
```

`create policy`	Write down a rule and give it a name.
`on documents for select`	It applies to the documents table, whenever anything is read.
`auth.uid()`	Who is asking - taken from the signed token, so it cannot be faked.
`= user_id`	Only rows whose name tag matches that person come back.

Written once

The rule lives on the table, not in the app.

Every page, every query, every future feature gets it automatically.

There are matching rules for writing

Creating, changing and deleting each have their own version, so you cannot write a row onto somebody else's name tag either.

UPLOADED DOCUMENTS

Private lockers

Files are kept separately

A passport scan does not go into the database. It goes into file storage, and the database records only where it was put.

Each person effectively gets their own locker, named after their account.

The rule: you may open your own locker only.

What the store refuses

Files larger than 10 MB

Anything that is not a PDF, JPEG or PNG

Any attempt to read or write inside someone else's locker

These are checked by the store itself, not by the upload screen.

Why the last line matters

A check on the upload screen is easy to skip - you can send a file to the store without ever opening our page. Because the store enforces the limits itself, skipping the screen achieves nothing.

OPENING A DOCUMENT

A ticket that expires

No public address

Your documents have no ordinary web address.

There is no link to guess, share by accident, or find in a search engine.

Click View

The backend checks it really is your file, then creates a one-off address for it.

That address works for 60 seconds.

Then it stops

After a minute the address is dead.

Even if it were copied or forwarded, it opens nothing.

The cloakroom ticket

You are not given a key to the cloakroom. You are given a ticket for one coat, and the ticket is only good for this evening.

THE QUESTION PEOPLE ASK

Is it safe to put a key in the browser?

There are two keys

The public key - in the app, readable by anyone. It says which project to talk to, and nothing more.

The secret key - never in the app, never in the code we publish. It ignores every rule and can read everything, so it belongs only on a server nobody else can reach.

Why the public one is fine

On its own it grants nothing.

It is a library card. Holding one lets you walk in and ask - it does not let you take other people's books off the shelf.

The rules on the tables decide what any given person actually receives.

The one rule never to break

The secret key must never be put anywhere the browser can see it. That single mistake would undo every protection described in this deck, because that key is designed to ignore them.

PUTTING IT TOGETHER

What happens when you open your Documents page

1	**Are you signed in?**	If not, you are sent to the login screen and nothing is requested.
2	**The page asks**	It sends one question to the database: give me my documents.
3	**The token travels with it**	Attached automatically, saying who is asking.
4	**The rule applies**	The database quietly narrows the question to rows carrying your name tag.
5	**Your rows come back**	Yours, and only yours - even though everyone's sit in the same table.
6	**Files stay put**	Only when you click View is a 60-second address created for that one file.

Notice what is missing

At no point does our code say 'and only show mine'. It cannot forget to, because it was never its job.

HOW THE DATABASE WAS BUILT

Three files, run in order

0001 **Create the tables**

The four tables, and the standing instruction that makes a profile for every new account.

0002 **Add the rules**

Turn on the protection, write the policies, create the private file store.

0003 **Fill the marketplace**

Add the five service providers students can browse.

Why keep them as files

The database was not clicked together by hand. It is described in three text files kept alongside the code, so the whole thing can be rebuilt from scratch, on a new project, in the same order, with the same result.

PROOF, NOT PROMISES

The same question, asked twice

Asked while signed out

Request: list the marketplace services.

Answer: an empty list.

Not an error, not a warning - simply nothing. The database declines to say that any rows exist at all.

Asked while signed in

Request: list the marketplace services.

Answer: all five providers.

Same code. Same key. Same question.

The only thing that changed is who was asking - and that is the whole point.

This was measured on the live site

Not a diagram of how it ought to behave - an actual request made against the running system, twice, with and without a session.

WHAT THE BACKEND REFUSES

Even to the person it belongs to

You may not approve yourself

A student can upload a passport scan.

A student cannot mark that scan verified, and cannot mark their own identity as confirmed.

Who can

A reviewer, working on a server, using the secret key.

That is the one thing allowed to ignore the rules - which is exactly why it never goes near a browser.

Why design it this way

The entire value of a verified document is that somebody independent checked it. If the person who uploaded the file could also tick the box saying it had been checked, the tick would mean nothing at all. The database is the right place to enforce that, because it is the one part of the system a determined user cannot talk their way around.

IN SHORT

Six sentences

- The backend is where the information lives and where the rules are enforced.
- We did not build a server - we configured Supabase and described our rules to it.
- Every application and every document carries a tag saying whose it is.
- The database reads that tag before answering anything, so it cannot return the wrong person's rows.
- Uploaded files sit in private lockers and are opened only through addresses that expire.
- The key in the browser is safe precisely because it is powerless without those rules.

Questions?

If you remember one thing: the lock is on the vault, not on the sign pointing to it.

LIVE SITE

multilingual-student-platform-navig.vercel.app

SOURCE

github.com/MarcCput/Multilingual-Student-Platform-NavigSA-